23-10-2026

INSTITUTO TECNOLÓGICO SUPERIOR DE MISANTLA

MATERIA:

TALLER DE BASES DE DATOS

CATEDRÁTICO:

Dr. IRAHÁN OTONIEL JOSÉ GUZMÁN

SEMESTRE: 5°

GRUPO: 503

UNIDAD:

3

ACTIVIDAD:

INVESTIGACIÓN DOCUMENTAL: TRANSACCIONES EN BASES DE DATOS

ALUMNOS:

SALOMÓN HERNÁNDEZ CANDANEDO – 152T0024

MARTINEZ MENDOZA KARLO DANIEL - 222T0113

SANDOVAL DORANTES ALEXANDRA JULIETH -242T0444

Introducción

Las bases de datos nos ayudan a almacenar información, de productos o cualquier tipo de inventarios y con ellas podemos ingresar, consultar, modificar e incluso eliminar datos.

Por ejemplo, en una tienda, cuando se realiza una venta debe guardar el detalle de la operación y descontarlos del inventario, si no se realiza correctamente la operación pueden existir contradictorias a la hora de consultar información en la BDD.

Las transacciones permiten agrupar operaciones relacionadas y tratarlas como una unidad lógica. Los sistemas gestores de bases de datos (SGBD) ofrecen mecanismos para confirmar cambios o deshacerlos, además de reglas de aislamiento que controlan cómo interactúan las operaciones concurrentes. En este reporte se explican las propiedades ACID, la consistencia, los niveles de aislamiento y el uso de START TRANSACTION, COMMIT y ROLLBACK. Finalmente, se presenta una práctica en MySQL con InnoDB.

Objetivo general

Comprender el funcionamiento de las transacciones en los sistemas gestores de bases de datos, sus propiedades, los niveles de aislamiento y los mecanismos para confirmar o deshacer operaciones, mediante una investigación documental y una práctica en MySQL.

Objetivos específicos

Definir qué es una transacción y explicar su importancia en aplicaciones reales.

Describir las propiedades ACID y los riesgos de no cumplirlas.

Distinguir la consistencia de los datos, las restricciones de integridad y los modelos de consistencia fuerte y eventual.

Comparar los niveles de aislamiento y reconocer los principales problemas de concurrencia.

Aplicar START TRANSACTION, COMMIT y ROLLBACK en una base de datos de ventas.

Verificar mediante consultas el estado de los datos antes y después de confirmar o deshacer una transacción.

Concepto e importancia y ciclo de vida de una transacción

Una transacción es un conjunto de operaciones de base de datos que se consideran una sola unidad lógica de trabajo. Puede incluir consultas, inserciones, actualizaciones y eliminaciones. El resultado esperado es que el conjunto se confirme completamente o que sus modificaciones se reviertan cuando sea necesario. En MySQL, START TRANSACTION inicia explícitamente la unidad de trabajo y COMMIT o ROLLBACK la termina (Oracle, s. f.-a).

Protegen la integridad: evitan que una operación compuesta quede aplicada solo a medias.

Controlan la concurrencia: permiten que múltiples usuarios trabajen sin interferencias indebidas.

Facilitan la recuperación: cuando una transacción falla antes de confirmarse, sus cambios pueden deshacerse.

Aportan confianza: los resultados confirmados se conservan conforme a las garantías del gestor y su configuración.

Una transacción suele tener un inicio, una secuencia de lecturas o escrituras y una decisión final. Si todas las validaciones se cumplen, se confirma; si ocurre un error o la aplicación detecta una condición inválida, se cancela. Algunos gestores también permiten puntos de recuperación parciales con SAVEPOINT, aunque no son necesarios para la práctica básica.

Etapa	Qué ocurre	Ejemplo de venta
Inicio	Se abre la transacción.	START TRANSACTION;
Lectura y validación	Se revisan existencias, precios o reglas.	Consultar si hay al menos 2 helados.
Operaciones	Se ejecutan cambios relacionados.	Descontar 2 unidades e insertar la venta.
Decisión	Se confirma o se cancela.	COMMIT si todo fue correcto; ROLLBACK si no.
Finalización	Se libera el control transaccional correspondiente.	Consultar los datos finales.

Ejemplos de situaciones reales

Transferencia bancaria: se descuenta dinero de una cuenta y se acredita a otra. Si solo se descuenta, se pierde dinero; si solo se acredita, se crea dinero. Las dos operaciones deben formar una unidad lógica.

Venta de una tienda: se registra la venta y se descuentan las unidades del inventario. Ambas deben corresponder a la misma operación.

Reserva de boletos: se reserva un asiento y se registra el pago o la intención de pago. El sistema debe evitar que dos personas obtengan el mismo asiento.

Inventario: una devolución puede aumentar existencias y generar un movimiento de entrada. Debe mantenerse el historial y el saldo coherentes.

Propiedades ACID

ACID es un conjunto de cuatro propiedades que describen las garantías clásicas de las transacciones: atomicidad, consistencia, aislamiento y durabilidad. No son sinónimos; cada una aborda un riesgo distinto.

Atomicidad: Significa que las operaciones de una transacción se tratan como un todo, se confirman juntas o se deshacen juntas. En una transferencia, el débito y el crédito no deben quedar separados.

Si no se cumple, una falla puede dejar el saldo de una cuenta reducido sin aumentar el saldo de la otra. En MySQL, el uso de tablas InnoDB permite revertir las modificaciones transaccionales que aún no se han confirmado.

Consistencia: Significa que una transacción válida debe llevar la base de un estado que cumple las reglas a otro estado que también las cumple. Las restricciones como PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK ayudan a impedir estados inválidos. La aplicación también debe comprobar reglas de negocio que no se expresan con una restricción simple.

Ejemplo: una existencia no debe ser negativa. Si la regla no se valida, una venta concurrente podría intentar vender más unidades de las disponibles.

Aislamiento: Significa que las transacciones concurrentes no deben interferir de manera que produzcan resultados incorrectos. El nivel de aislamiento determina qué efectos de otras transacciones pueden observarse y qué anomalías se evitan.

Si no se controla, dos cajeros podrían leer existencia 1 y ambos vender la última unidad. El control puede apoyarse en bloqueos, versiones de filas o mecanismos de detección de conflictos, según el SGBD.

Durabilidad: Una vez que COMMIT termina correctamente, los cambios confirmados deben sobrevivir a fallos conforme a las garantías del gestor y la configuración de almacenamiento. La durabilidad depende también de la configuración del servidor, el sistema operativo y el almacenamiento.

Si no se cumple, una venta confirmada podría desaparecer después de un reinicio inesperado. Por eso se requieren mecanismos de registro, recuperación y copias de seguridad; las copias no sustituyen a las transacciones, pero ayudan ante otros tipos de pérdida.

Propiedad	Pregunta que responde	Ejemplo	Riesgo si falla
Atomicidad	¿Se aplica todo o nada?	Débito y crédito de una transferencia.	Dinero descontado sin acreditarse.
Consistencia	¿Se respetan las reglas?	Existencia nunca negativa.	Inventario con valores inválidos.
Aislamiento	¿Cómo conviven transacciones simultáneas?	Dos cajeros venden el mismo artículo.	Sobreventa o lecturas engañosas.
Durabilidad	¿Se conservan los cambios confirmados?	Venta confirmada tras reinicio.	Pérdida de operaciones confirmadas.

Consistencia de los datos

La consistencia de los datos es el grado en que la información cumple las reglas, relaciones y condiciones que el sistema considera válidas. Por ejemplo, cada detalle de venta debe referirse a una venta existente, el precio no debe ser negativo y la cantidad vendida debe ser positiva.

Restricciones de integridad

Restricción	Función	Ejemplo
PRIMARY KEY	Identifica cada fila sin duplicados ni valores nulos.	id_producto como clave primaria.
FOREIGN KEY	Mantiene relaciones válidas entre tablas.	ventas. id_producto referencia productos.
NOT NULL	Impide que un campo obligatorio quede vacío.	nombre del producto.
UNIQUE	Evita duplicados en una columna o combinación.	correo o código de producto.
CHECK	Valida una condición sobre los valores.	existencia >= 0; cantidad > 0.
DEFAULT	Asigna un valor cuando no se proporciona otro.	fecha de venta actual.

Estas restricciones son importantes porque se aplican en la base de datos, no solamente en la interfaz. Sin embargo, una regla de negocio como “no permitir vender más unidades de las existentes” requiere diseñar correctamente la actualización y verificar cuántas filas fueron afectadas, especialmente cuando existen operaciones concurrentes.

Consistencia fuerte y consistencia eventual

En sistemas distribuidos, los datos pueden mantenerse en varias réplicas. La consistencia fuerte suele significar que una lectura posterior a una escritura confirmada observa el valor más reciente de acuerdo con el modelo ofrecido por el sistema.

La consistencia eventual permite que, durante un periodo, distintas réplicas muestren valores diferentes; si dejan de producirse nuevas escrituras y la replicación progresa, las réplicas convergen.

Aspecto	Consistencia fuerte	Consistencia eventual
Lectura después de escribir	Ofrece la garantía de observar el dato actualizado según el modelo.	Puede devolver temporalmente un dato anterior.
Latencia/disponibilidad	Puede requerir coordinación entre nodos y aumentar la latencia.	Puede responder sin esperar a que todas las réplicas converjan.
Uso típico	Saldos bancarios, reservas críticas, inventario que no debe sobrevenderse.	Contadores de visitas, métricas, algunos perfiles o catálogos replicados.
Riesgo principal	Mayor coordinación o bloqueos según la arquitectura.	Usuarios pueden observar temporalmente versiones distintas.

Los términos se relacionan, pero no son idénticos: ACID describe propiedades de transacciones; los modelos fuerte y eventual describen garantías de lectura y actualización, especialmente en sistemas replicados y distribuidos. Un sistema puede combinar transacciones ACID en una base local con replicación eventual hacia otros servicios.

Ejemplo: al vender el último helado, la base principal debe evitar que se registren dos ventas incompatibles. Un panel de estadísticas replicado podría actualizarse unos segundos después sin afectar la venta. En cambio, un sistema de reservas no debería confirmar dos veces el mismo asiento por depender de una réplica atrasada.

Niveles de aislamiento y problemas de concurrencia

Problemas principales:

Lectura sucia (dirty read): Una transacción lee un cambio que otra transacción todavía no ha confirmado. Si la segunda termina en ROLLBACK, la primera utilizó un valor que nunca llegó a ser definitivo.

Lectura no repetible (non-repeatable read): Una transacción lee una fila y, al leerla de nuevo dentro de la misma transacción, encuentra un valor diferente porque otra transacción modificó y confirmó esa fila.

Lectura fantasma (phantom read): Una consulta que busca un conjunto de filas se ejecuta dos veces y devuelve un conjunto distinto porque otra transacción insertó, eliminó o modificó filas que cumplen la condición.

Tabla comparativa

Nivel	Lectura sucia	Lectura no repetible	Lectura fantasma	Comentario
READ UNCOMMITTED	Puede permitirla	Puede ocurrir	Puede ocurrir	Menor aislamiento; no conviene para datos críticos.
READ COMMITTED	La evita	Puede ocurrir	Puede ocurrir	Cada lectura ve datos ya confirmados al comenzar la consulta.
REPEATABLE READ	La evita	La evita	Depende del SGBD	En InnoDB el comportamiento de lecturas consistentes reduce las anomalías fantasmas; PostgreSQL también las evita en este nivel.
SERIALIZABLE	La evita	La evita	La evita	Garantiza un resultado equivalente a ejecutar transacciones en serie; puede provocar bloqueos o abortos que deben reintentarse.

La tabla resume el comportamiento habitual definido por el estándar, pero existen diferencias entre implementaciones. PostgreSQL trata READ UNCOMMITTED como READ COMMITTED y su REPEATABLE READ evita también las lecturas fantasmas. En InnoDB, el nivel predeterminado es REPEATABLE READ y se combinan control de versiones y bloqueos.

Implementación en MySQL/InnoDB

```
-- Consultar el nivel actual de la sesión
SELECT @@transaction_isolation;

-- Definir el nivel para la siguiente transacción
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT * FROM productos;
COMMIT;

-- También se puede usar:
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
-- consultas y operaciones
COMMIT;
```

InnoDB soporta READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ y SERIALIZABLE; su valor predeterminado es REPEATABLE READ. La aplicación puede seleccionar otro nivel según el riesgo de anomalías y el costo de concurrencia. La elección del nivel no reemplaza las restricciones de integridad ni las validaciones del negocio.

Implementación en PostgreSQL

```
SHOW transaction_isolation;

BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT * FROM productos;
COMMIT;
```

PostgreSQL usa MVCC (control de concurrencia multiversión) para proporcionar vistas consistentes de los datos. En SERIALIZABLE puede detectar conflictos y cancelar una transacción; la aplicación debe estar preparada para repetir la transacción completa cuando se presente un error de serialización.

COMMIT, ROLLBACK y manejo de errores

START TRANSACTION: Inicia explícitamente una transacción de varias sentencias. En MySQL, el modo autocommit está habilitado por defecto; fuera de una transacción explícita, cada sentencia exitosa suele confirmarse por separado.

Por eso, para agrupar operaciones relacionadas se inicia una transacción y se termina con COMMIT o ROLLBACK.

COMMIT: Confirma la transacción actual y hace permanentes sus modificaciones. Debe utilizarse después de verificar que todas las operaciones necesarias se ejecutaron correctamente.

ROLLBACK: Revierte las modificaciones realizadas en la transacción activa desde su inicio o desde el punto de recuperación correspondiente. No puede deshacer cambios que ya fueron confirmados por un COMMIT anterior.

Instrucción	Propósito	Cuando usarla
START TRANSACTION	Comenzar una unidad de trabajo explícita.	Antes de agrupar varias operaciones relacionadas.
COMMIT	Confirmar los cambios.	Cuando las operaciones y validaciones terminan correctamente.
ROLLBACK	Deshacer los cambios pendientes.	Ante un error, validación fallida o cancelación.

Ejemplo de confirmación

```
START TRANSACTION;
UPDATE productos
SET existencia = existencia - 2
WHERE id_producto = 1 AND existencia >= 2;

INSERT INTO ventas (id_producto, cantidad, total)
SELECT 1, 2, precio * 2
FROM productos
WHERE id_producto = 1;

COMMIT;
```

El ejemplo supone que la aplicación verificó que el UPDATE afectó exactamente una fila antes de insertar la venta. En una aplicación real se debe manejar ese resultado para no registrar una venta si no se descontó inventario.

Ejemplo de reversión

```
START TRANSACTION;
UPDATE productos
SET existencia = existencia - 3
WHERE id_producto = 1 AND existencia >= 3;

INSERT INTO ventas (id_producto, cantidad, total)
SELECT 1, 3, precio * 3
FROM productos
WHERE id_producto = 1;

ROLLBACK;
```

La segunda prueba simula una operación que finalmente se cancela. Después de ROLLBACK, la existencia y el historial deben regresar al estado confirmado anterior, siempre que las tablas sean transaccionales y no se haya ejecutado una confirmación intermedia.

¿Qué sucede cuando ocurre un error?

El efecto depende del tipo de error y del SGBD. En MySQL, un error de una sentencia puede cancelar esa sentencia sin necesariamente cancelar toda la transacción; por ello, el programa debe detectar el error y decidir si ejecuta ROLLBACK. Algunos errores graves, como ciertos tiempos de espera o deadlocks, pueden hacer que InnoDB revierta la transacción. No se debe asumir que todos los errores revierten automáticamente el conjunto completo.

En código de aplicación se recomienda: iniciar la transacción; ejecutar operaciones; verificar filas afectadas y reglas; confirmar solo si todo salió bien; y, ante una excepción, intentar ROLLBACK y registrar el error. Las sentencias de definición de datos (por ejemplo, CREATE TABLE) pueden provocar commits implícitos en MySQL, por lo que la práctica separa la creación de tablas de las pruebas transaccionales.

Práctica obligatoria: Transacciones en MySQL

Comprobar experimentalmente la diferencia entre COMMIT y ROLLBACK mediante una venta de helados. La práctica utiliza las tablas productos y ventas con motor InnoDB.

Requisitos

- MySQL Server o MariaDB y un cliente SQL, por ejemplo, MySQL Workbench.
- Ejecutar el archivo practica_transacciones_mysql.sql incluido con este reporte.
- Ejecutar el script en una sola sesión y observar los resultados de los SELECT.
- Tomar capturas propias de los resultados antes, durante y después de cada prueba.

Estructura de las tablas

```
CREATE TABLE productos (
id_producto INT PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,
precio DECIMAL(10,2) NOT NULL CHECK (precio >= 0),
existencia INT NOT NULL CHECK (existencia >= 0)
) ENGINE=InnoDB;

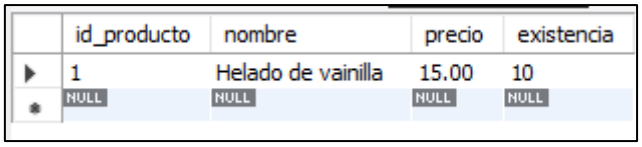
CREATE TABLE ventas (
id_venta INT AUTO_INCREMENT PRIMARY KEY,
id_producto INT NOT NULL,
cantidad INT NOT NULL CHECK (cantidad > 0),
total DECIMAL(10,2) NOT NULL CHECK (total >= 0),
fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
) ENGINE=InnoDB;
```

Insertamos datos a nuestra tabla productos

```
INSERT INTO productos
(id_producto, nombre, precio, existencia)
VALUES (1, 'Helado de vainilla', 15.00, 10);

SELECT * FROM productos;
SELECT * FROM ventas;
```

Mostramos el resultado de los SELECT en la tabla productos se insertó un producto.



id_producto	nombre	precio	existencia
1	Helado de vainilla	15.00	10
NULL	NULL	NULL	NULL

La tabla ventas se muestra vacía porque no le hemos metido datos.



id_venta	id_producto	cantidad	total	fecha
NULL	NULL	NULL	NULL	NULL

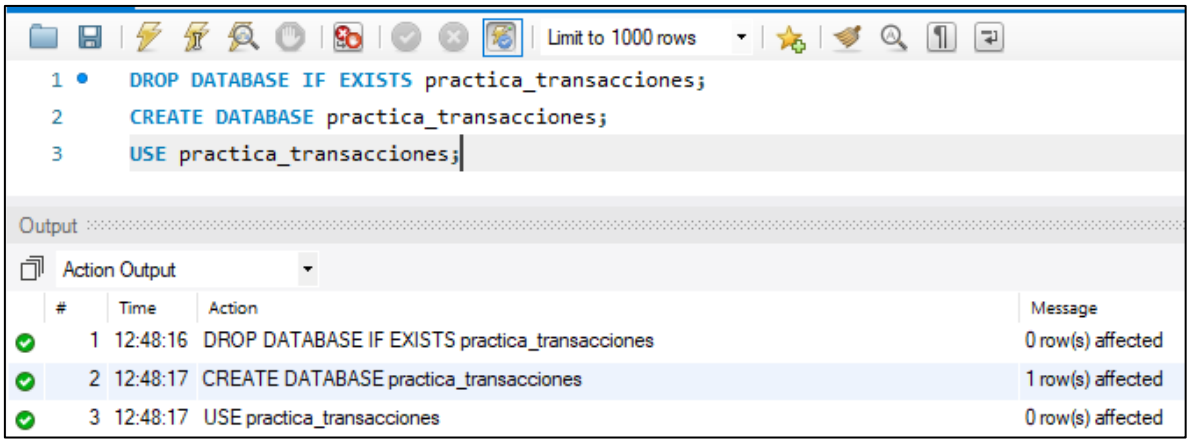
COMMIT: Se descuenta una cantidad de 2 unidades y se registra una venta por $30.00. Antes de confirmar, las consultas en la misma sesión pueden mostrar los cambios provisionales. Después de COMMIT, los cambios quedan confirmados.

Momento	Existencia esperada	Número de ventas	Interpretación
Antes	10	0	Estado inicial.
Dentro de la transacción	8	1	Cambios visibles en la sesión que los realizó.
Después de COMMIT	8	1	Cambios confirmados.

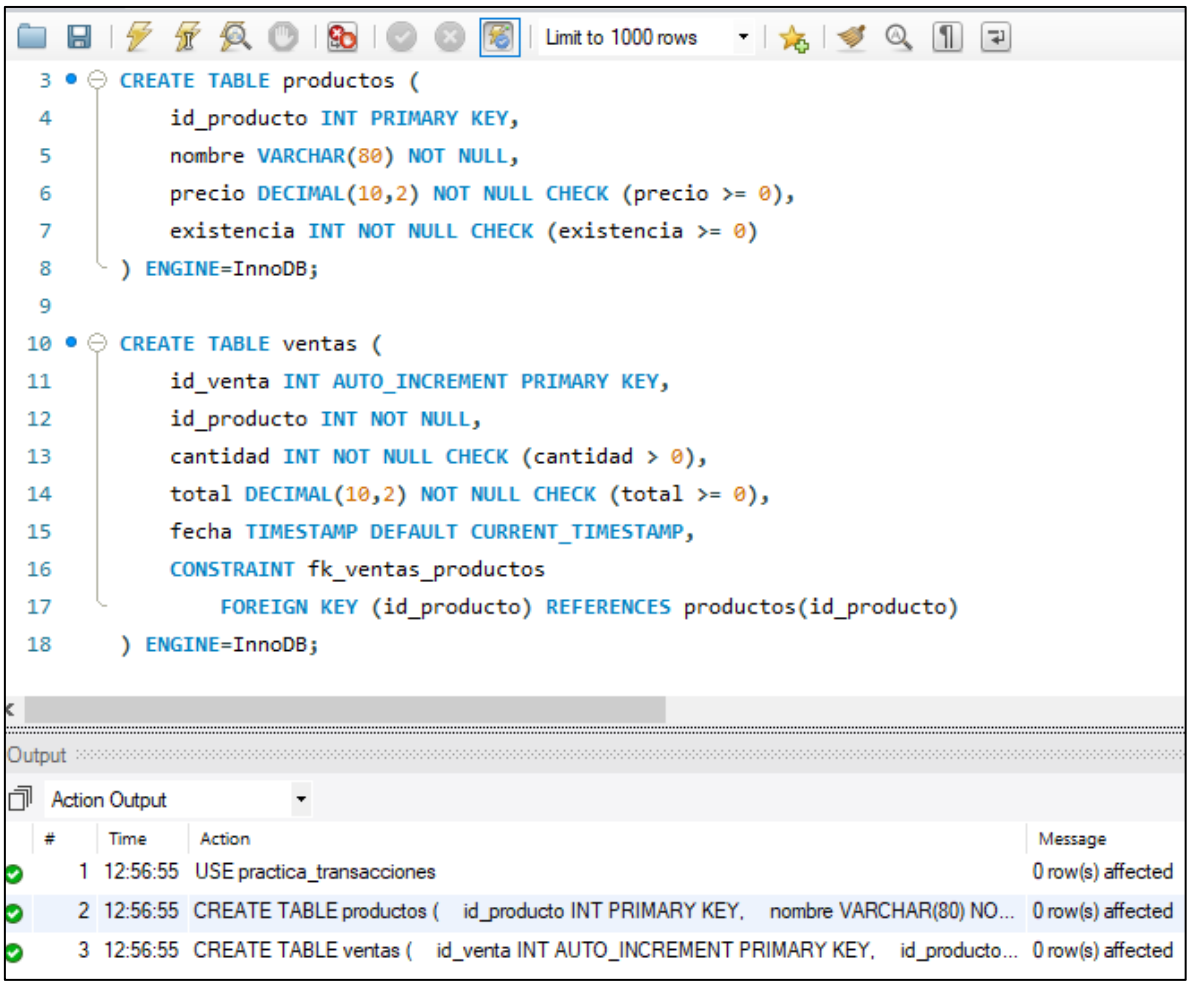
ROLLBACK: Partiendo del estado confirmado de la prueba 1, se intenta vender 3 unidades. Antes de ROLLBACK, la sesión debe mostrar existencia provisional 5 y dos ventas. Al ejecutar ROLLBACK, se revierte la segunda operación: la existencia vuelve a 8 y queda solamente la primera venta.

Momento	Existencia esperada	Número de ventas	Interpretación
Antes de la segunda prueba	8	1	Estado confirmado por la prueba 1.
Dentro de la transacción	5	2	Cambios provisionales de la segunda venta.
Después de ROLLBACK	8	1	La segunda venta se deshizo.

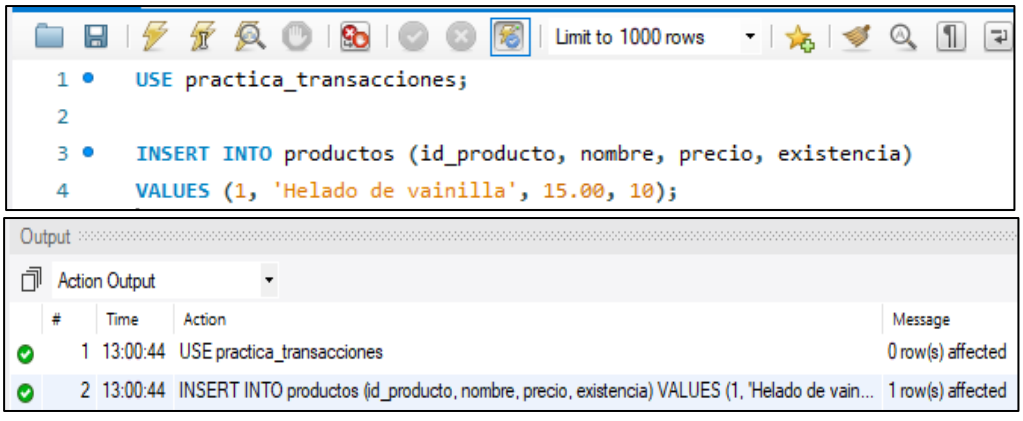
Evidencias de ejecución



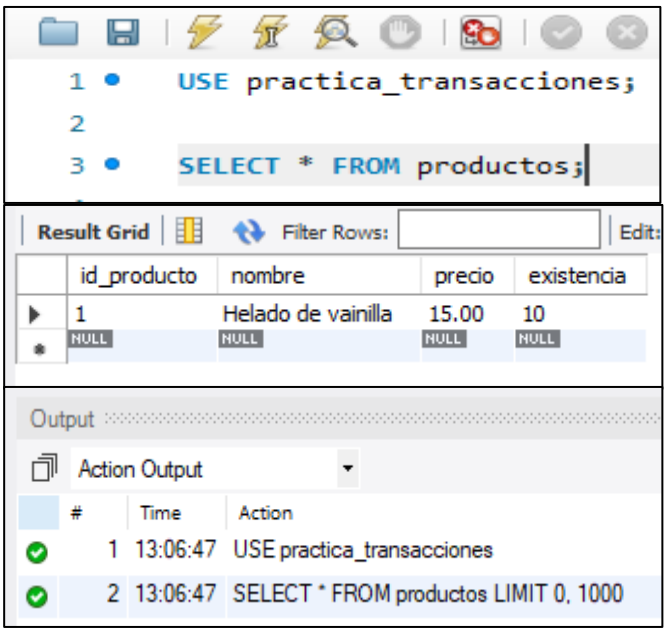
Creamos la BDD con el nombre practica_transacciones.



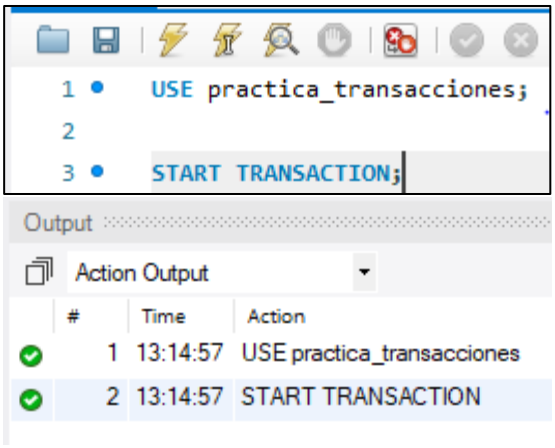
Creamos dos tablas que vamos a utilizar durante la practica una llamada productos y otra llamada ventas.



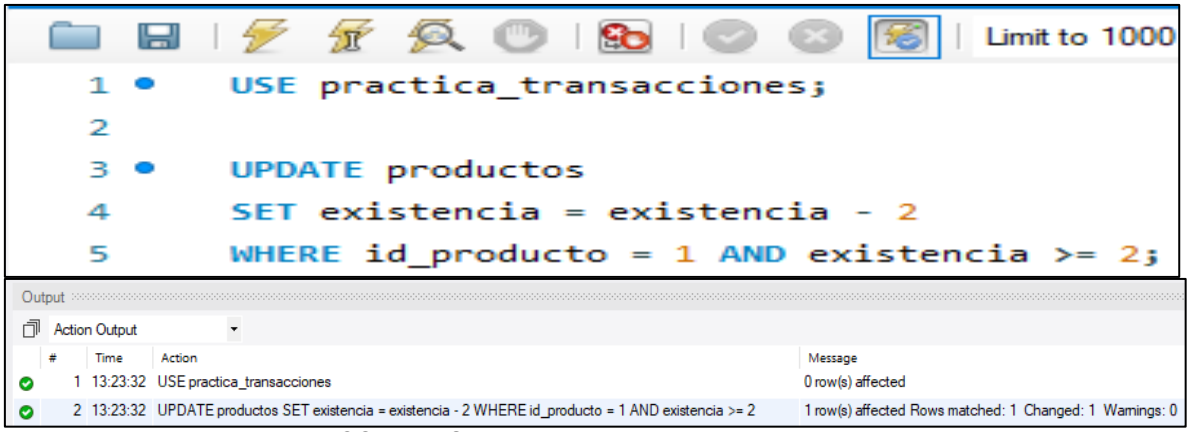
Insertamos un dato a la tabla productos.



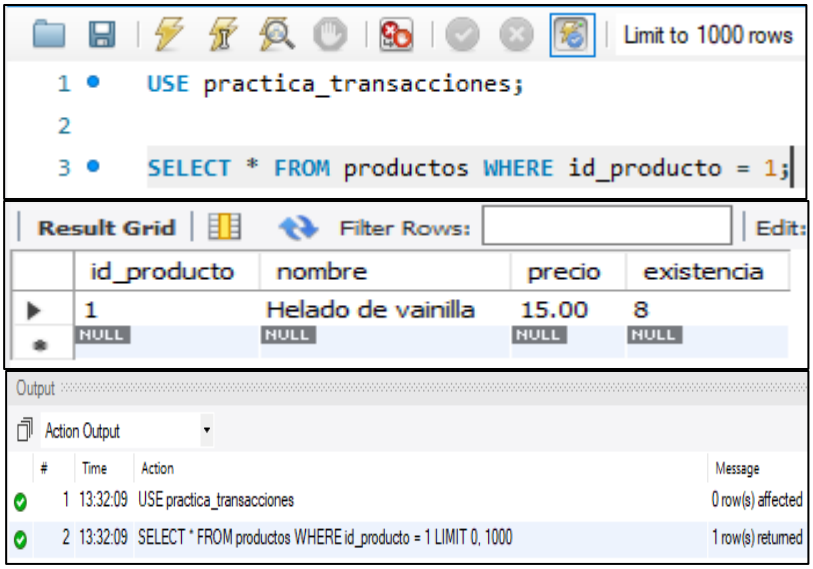
Realizamos un SELECT para comprobar que si se haya realizado la inserción a la tabla productos.

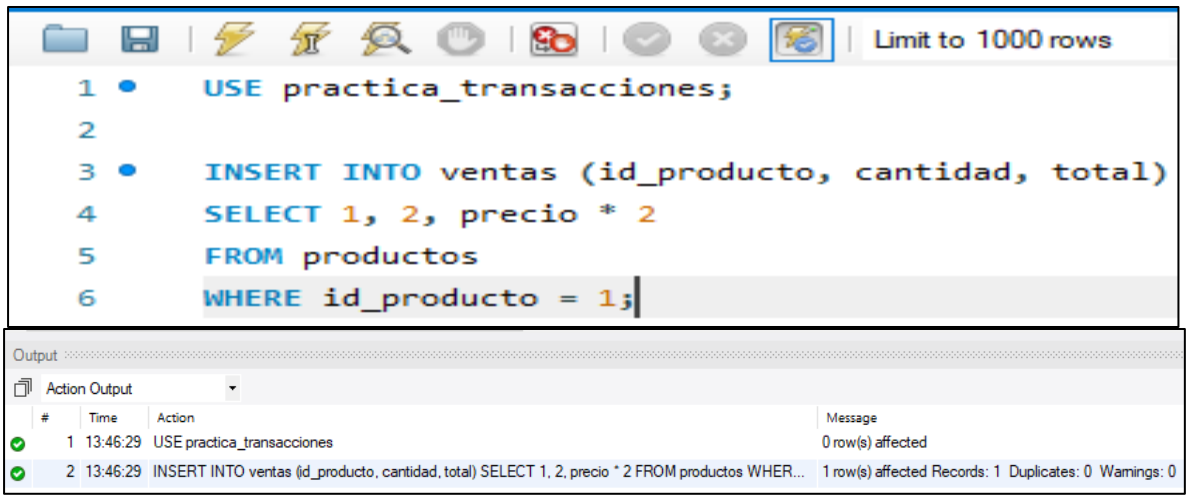


Iniciamos la Transacción.

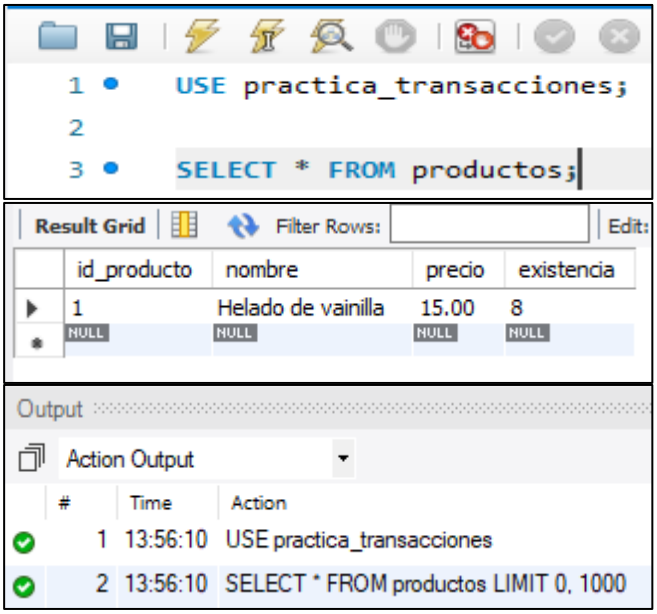


PRUEBA 1: COMMIT. Se descuentan 2 helados y se registra la venta.

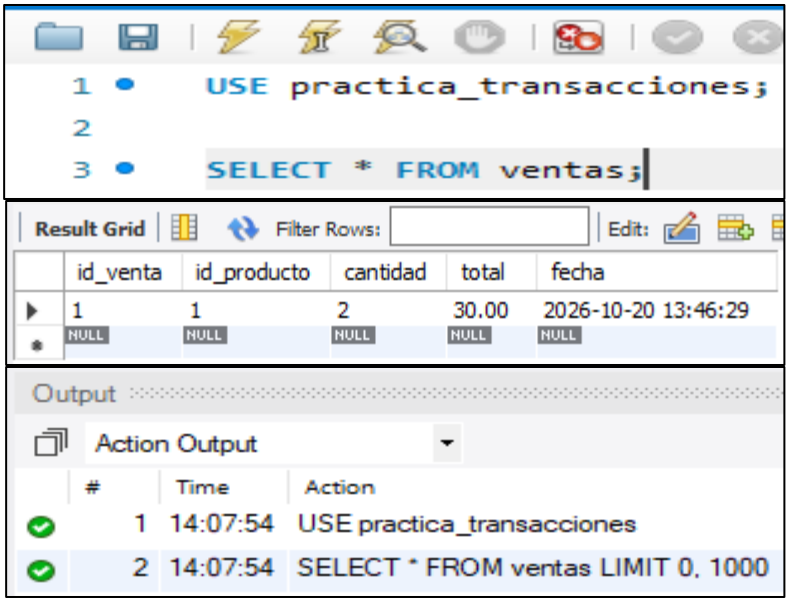




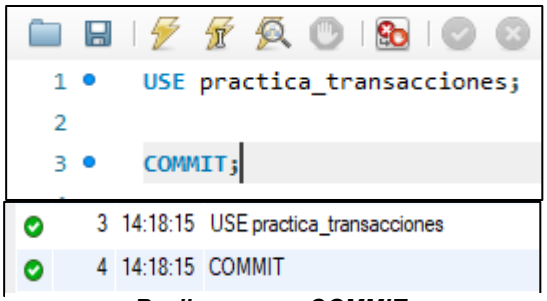
Registramos una venta de 2 unidades en la tabla ventas.



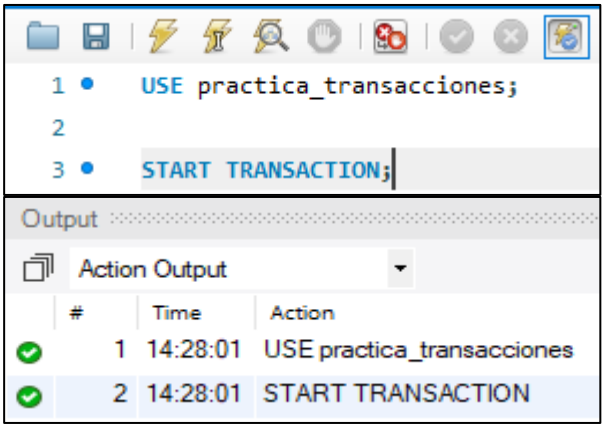
Hacemos un SELECT a la tabla productos se realizó la venta, pero no se modificó la tabla.



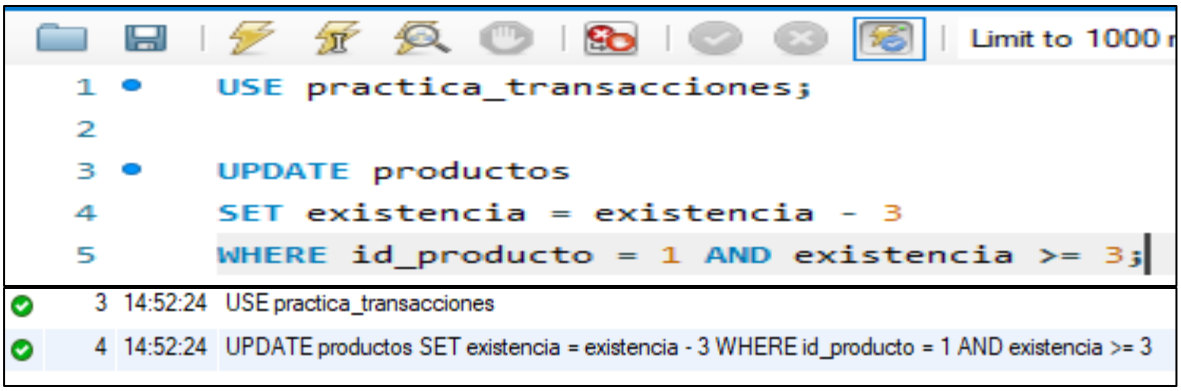
Verificamos que se realizó la venta correctamente y se registró en la tabla ventas.



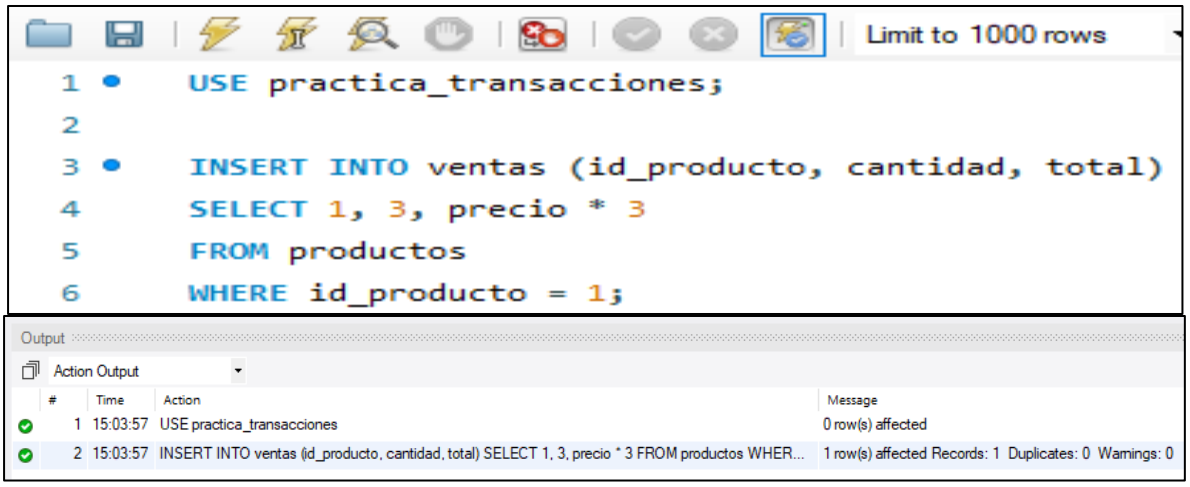
Realizamos un COMMIT.



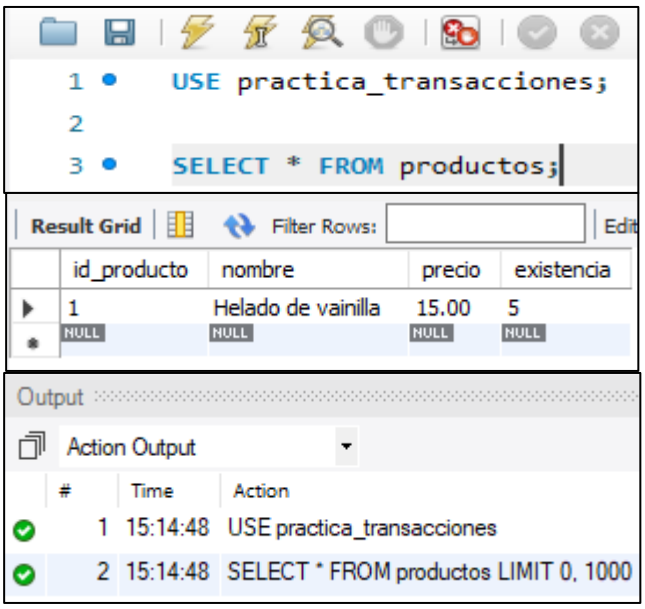
PRUEBA 2: ROLLBACK. Se simula otra venta, pero se cancela.



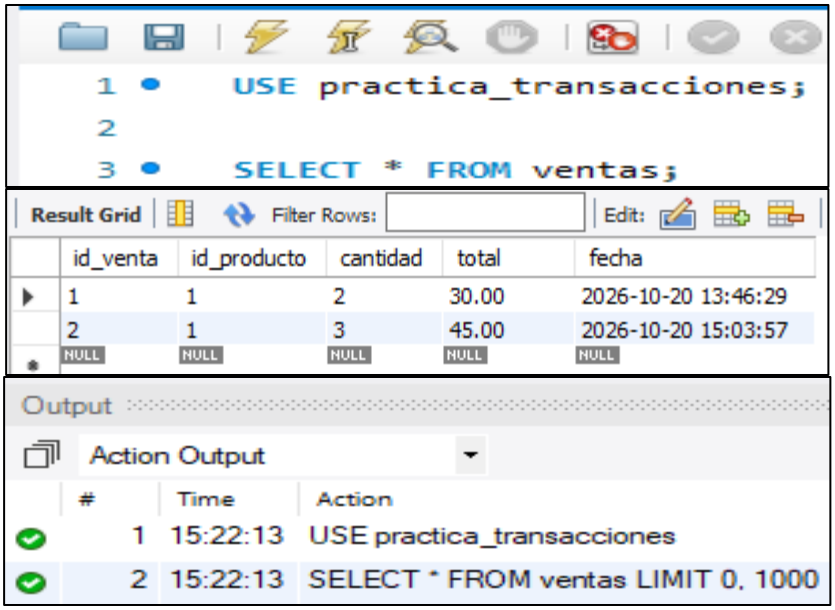
Descontamos 3 unidades de la existencia del producto que existe en la taba productos.



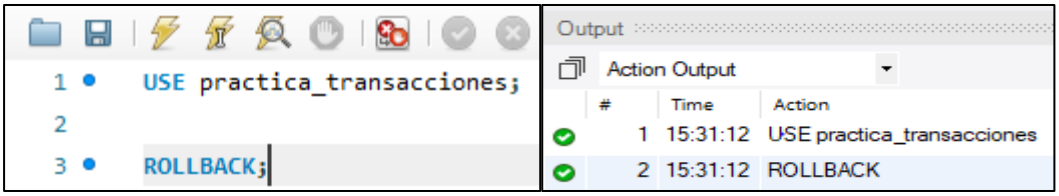
Se registra una venta de 3 unidades del producto con ID = 1 y calcula el total multiplicándolo por 3.



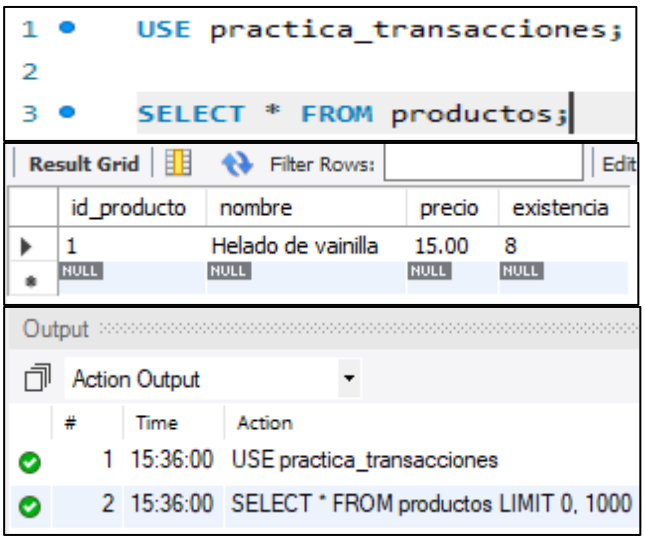
Ejecutamos un SELECT para verificar que el UPDATE que se ejecutó antes de esta consulta se realizó correctamente y se descontaron 3 unidades en existencia.



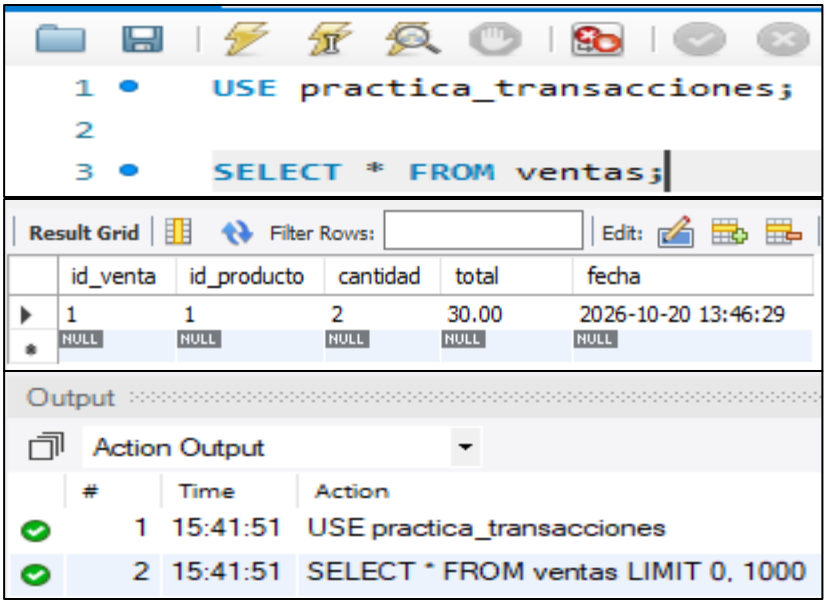
Ejecutamos un SELECT antes de realizar el ROLLBACK y verificamos que se haya realizado la venta correctamente.



Ejecutamos el ROLLBACK.



Ejecutamos un SELECT y confirmamos que después del ROLLBACK los datos volvieron a como estaban antes en la tabla productos.



Ejecutamos un SELECT y confirmamos que después del ROLLBACK los datos volvieron a como estaban antes en la tabla ventas.

Análisis de resultados

La prueba con COMMIT confirma que las operaciones relacionadas quedan registradas juntas: la existencia disminuye de 10 a 8 y aparece una venta por $30.00. La prueba con ROLLBACK demuestra que los cambios de una transacción todavía activa pueden cancelarse: la existencia retorna a 8 y la venta provisional desaparece. El experimento permite observar la atomicidad en una situación sencilla y muestra por qué no se debe confirmar una transacción hasta validar todas sus operaciones.

Si los resultados difieren, se debe comprobar que el script se ejecutó en una sola conexión, que las tablas son InnoDB, que no se ejecutó COMMIT antes de ROLLBACK y que se está consultando la base de datos correcta.

Conclusiones personales

Las transacciones son esenciales porque agrupan operaciones que deben mantener una relación lógica. Comprendí que ACID ayuda a analizar cuatro riesgos diferentes: una operación incompleta, reglas incumplidas, interferencia entre usuarios y pérdida de cambios confirmados. También observé que COMMIT y ROLLBACK no son intercambiables: el primero confirma los cambios y el segundo cancela únicamente los que continúan pendientes.

La comparación de los niveles de aislamiento muestra que una mayor protección frente a anomalías puede implicar más coordinación, bloqueos o reintentos. Por eso, el nivel se debe elegir según las necesidades del sistema y no solo por buscar el nivel más alto. Las restricciones de integridad, la validación de la aplicación y el diseño correcto de las transacciones trabajan en conjunto para conservar datos confiables.

La práctica de ventas permite relacionar la teoría con un caso concreto. La evidencia más importante es comparar los datos antes y después de cada operación: después de COMMIT se conserva la venta, mientras que después de ROLLBACK se recupera el estado confirmado previo. Las capturas finales deberán corresponder a la ejecución real realizada por el estudiante.

Referencias

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Oracle. (s. f.-a). *START TRANSACTION, COMMIT, and ROLLBACK statements*. MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/commit.html

Oracle. (s. f.-b). *InnoDB transaction isolation levels*. MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html

PostgreSQL Global Development Group. (s. f.-a). *Introduction to MVCC*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/mvcc-intro.html

PostgreSQL Global Development Group. (s. f.-b). *Transaction isolation*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/transaction-iso.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw Hill.